

Bird Sort 2 - Color Puzzle

Powerpoint checkpoint 1

Step 1: Understanding the Game (Bird Sort 2 - Color Puzzle)

The Bird Sort 2 - Color Puzzle is a solitaire sorting puzzle where the player must arrange birds of different colors into separate branches.

The game mechanics are similar to the Water Sort Puzzle Game, where birds can only be moved according to specific rules.

Game Elements:

- **Board Representation:** A set of branches, each containing a stack of birds in different colors.
 - **Valid Moves:**
 - A bird can only be moved to another branch if:
 - The branch is not full.
 - The top bird of the destination branch is the same color as the bird being moved.
 - The branch is empty (any bird can be placed in an empty branch).
 - **Goal:** Arrange the birds so that each branch contains only birds of a single color.
-

Step 2: Defining the AI Approach

Since the AI must solve this puzzle using search methods, we need to define:

1. State Representation:

- The game state must encode the current arrangement of birds in the branches.
- A possible representation is a list of lists, where each sublist represents a branch and contains color-coded bird identifiers.

2. State Transitions:

- Every valid move results in a new state.
- A function should generate possible moves and apply them to create new states.

3. Goal State Check:

- The game is solved when all branches contain birds of only one color.

4. Search Algorithms:

- **Uninformed Search Methods:**
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)
 - Iterative Deepening Search
 - Uniform Cost Search (UCS)

- **Heuristic Search Methods:**
 - Greedy Best-First Search
 - A* Algorithm
 - Weighted A*

5. Heuristic Functions:

- Possible heuristics include:
 - The number of branches that already contain sorted birds.
 - The number of misplaced birds.
 - The number of empty branches available.

Step 3: Basic Implementation Plan

1. Implement the Board Representation:

- Use a nested list to represent the branches and the birds in them.
- Create functions to initialize the board from a text file.

2. Implement Move Validation and State Generation:

- Develop a function to check if a move is valid.
- Implement a function that generates all possible next states from the current state.

3. Implement Basic Human-Playable Mode:

- Allow users to manually move birds between branches.
- Display the board state in the console.

4. Implement Search Algorithms One by One:

- Implement BFS, DFS, and other uninformed search methods.
- Implement heuristic-based methods (A*, Weighted A*).
- Compare search methods based on efficiency (time, memory, solution quality).

5. Optimize Heuristic Search:

- Experiment with different heuristic functions to improve the efficiency of A* search.
- Implement priority queue-based state exploration.

6. Add Graphical Interface:

- Use `pygame` or `tkinter` to create a visual representation of the puzzle.
- Implement a mode where the AI solves the puzzle visually.
- Add a hint system for human players.

7. Final Testing and Optimization:

- Test the application with multiple puzzles of varying difficulty.
- Record performance metrics (time, memory usage, solution length).
- Optimize code for efficiency and clarity.

Step 4: Choosing Python Libraries

- `heapq` – For priority queue management in A* and Weighted A*.
 - `pygame` or `tkinter` – For GUI visualization of the puzzle.
 - `pickle` or `json` – For saving and loading game states.
 - `psutil` – For tracking memory usage during search execution (optional).
-